

codeHive ACADEMY

Insertion Sort

THE CARD PLAYER'S ALGORITHM

DSA Series 2026

Module 06: Sorting Logic

What is Insertion Sort?

Insertion Sort is an intuitive algorithm that builds a sorted list one item at a time. It works similarly to the way you might sort playing cards in your hands: you pick up one card from the "unsorted" pile and insert it into its correct position in the "sorted" hand.

How it Works

1. Assume the first element is already **sorted**.
2. Take the next element from the **unsorted** part.
3. Compare it with elements in the sorted part and move those elements until you find the correct "insertion point."
4. Insert the element and repeat until the array is fully sorted.

Manual Walkthrough

Let's trace the array: [7, 12, 9, 11, 3]

Step 1: Start with 7. It's the only item in the "sorted" section.



Step 2: Look at 12 . $12 > 7$, so it stays put.



Step 3: Look at 9 . $9 < 12$, so shift 12. $9 > 7$, so insert between 7 and 12.



Step 4: Look at 11 . $11 < 12$, so shift 12 and insert 11.



Step 5: Look at 3 . Shift everything! 3 is the new front.



Basic Implementation

This version uses `pop()` and `insert()`. While intuitive, it triggers many "under-the-hood" memory operations.

PYTHON: INTUITIVE

```
my_array = [64, 34, 25, 12, 22, 11, 90, 5]

n = len(my_array)
for i in range(1, n):
    insert_index = i
    current_value = my_array.pop(i)
    # Find the correct spot
    for j in range(i-1, -1, -1):
        if my_array[j] > current_value:
            insert_index = j
    my_array.insert(insert_index, current_value)

print("Sorted array:", my_array)
```

The Performance Trap

High-level languages like Python hide the "Shifting Problem." Every time you `pop()` or `insert()`, the computer has to physically move every subsequent element in memory to close the gap or make room. This is very slow for large arrays.

Improved Implementation

The solution is to manually shift only the necessary elements using variable reassignment, rather than removing/inserting entire blocks of memory.

PYTHON: OPTIMIZED

```
my_array = [64, 34, 25, 12, 22, 11, 90, 5]

n = len(my_array)
for i in range(1, n):
    insert_index = i
    current_value = my_array[i]
    for j in range(i-1, -1, -1):
        if my_array[j] > current_value:
            # Shift value up manually
            my_array[j+1] = my_array[j]
            insert_index = j
        else:
            break # Spot found!
    my_array[insert_index] = current_value

print("Sorted array:", my_array)
```

Why is this faster? We only shift elements that are actually larger than our `current_value`. Once we find an element that is smaller, we stop the inner loop immediately (`break`).

Expert Analysis: Time Complexity

Like Selection Sort and Bubble Sort, Insertion Sort belongs to the **Quadratic Time** family, but it performs remarkably well in specific scenarios.

Scenario	Complexity	Result
Best Case	$O(N)$	Array is already sorted.
Worst Case	$O(N^2)$	Array is in reverse order.
Average Case	$O(N^2)$	Standard unsorted data.

💡 codeHive Tip

Insertion Sort is often used in the "Real World" as the final step for algorithms like **Quicksort** or **Timsort** (Python's default sort). When the sub-arrays become very small (usually < 10 elements), Insertion Sort is actually faster than more complex algorithms because it has lower overhead.

Test Your Knowledge

Exercise: How many times must the outer loop run to sort an array with 10 elements using Insertion Sort?

Answer: 9 times (The first element is considered sorted initially).

codeHive

Ready to go faster? Up next: **QuickSort**.